

Day 8 — Sat 12 Sep

Countdown: 14 days to General English & GK

Method: start by recalling yesterday's topic with this page closed. Study the sections below, work every example by hand, then do 20 MCQs.

● GK — Polity I

Source: General/03_GK_CAPSULE.md

1.1 The Constitution — the frame

Intuition first. A constitution is just the rulebook that says who may do what, and what the state may never do to you. India's is the longest written constitution in the world because it tries to write *everything* down rather than leave it to convention.

Fact	Value
Constituent Assembly formed	1946 (under the Cabinet Mission Plan)
Chairman, Drafting Committee	Dr. B. R. Ambedkar
President of the Constituent Assembly	Dr. Rajendra Prasad
Adopted	26 November 1949 (Constitution Day / <i>Samvidhan Divas</i>)
Came into force	26 January 1950 (Republic Day)
Time taken	2 years, 11 months, 18 days
Borrowed features	Parliamentary system & rule of law — UK; Fundamental Rights & judicial review — USA; DPSP — Ireland; Emergency — Germany; Concurrent List & trade/commerce — Australia; Fundamental Duties & planning — USSR; amendment procedure — South Africa

MCQ trap: the Constitution was *adopted* on 26 Nov 1949 but *came into force* on 26 Jan 1950. Questions deliberately swap these two dates.

1.3 Fundamental Rights (Part III, Articles 12–35)

Intuition. These are the promises the state makes to *you* that a court can enforce. If the state breaks one, you can go straight to the Supreme Court (Art. 32) — which Ambedkar called "**the heart and soul of the Constitution.**"

#	Right	Articles	Core content
1	Right to Equality	14–18	Equality before law (14); no discrimination on religion/race/caste/sex/birth (15); equal opportunity in public employment (16); abolition of untouchability (17); abolition of titles (18)
2	Right to Freedom	19–22	Six freedoms — speech, assembly, association, movement, residence, profession (19); protection re conviction (20); protection of life & personal liberty (21) ; right to education 6–14 yrs (21A); protection against arrest (22)

3	Right against Exploitation	23–24	Ban on human trafficking & forced labour (23); ban on child labour under 14 in hazardous work (24)
4	Right to Freedom of Religion	25–28	Freedom of conscience & practice (25); manage religious affairs (26); no tax for religion (27); no religious instruction in wholly state-funded schools (28)
5	Cultural & Educational Rights	29–30	Protection of minorities' language/script/culture (29); minorities' right to run educational institutions (30)
6	Right to Constitutional Remedies	32	Right to move the SC for enforcement — the "heart and soul"

The five writs (under Art. 32 / 226):

Writ	Literal meaning	Used to...
Habeas Corpus	"to have the body"	Produce a detained person; check unlawful detention
Mandamus	"we command"	Order a public official to do their duty
Prohibition	—	Stop a lower court exceeding its jurisdiction (before)
Certiorari	"to be certified"	Quash a lower court's order (after)
Quo Warranto	"by what authority"	Question a person's right to hold a public office

MCQ traps:

- Right to Property was **removed** as a Fundamental Right by the **44th Amendment (1978)**; it is now a *legal* right under **Art. 300A**. Very frequently asked.
- Art. **20 and 21 cannot be suspended even during a National Emergency** (44th Amdt).

1.12 Constitutional & other key bodies

Body	Article	Role
Election Commission	324	Conducts elections; a multi-member body (CEC + ECs)
UPSC	315	Recruits for central services
CAG	148	Audits government accounts; "guardian of the public purse"
Finance Commission	280	Recommends Centre–State tax sharing (every 5 yrs)
Attorney General	76	Chief legal adviser to the Government of India
NITI Aayog	<i>(not constitutional; not statutory)</i>	Policy think-tank, replaced Planning Commission (2015)

MCQ trap: NITI Aayog and the Planning Commission are **executive** bodies (set up by resolution), **not** constitutional bodies. The Finance Commission **is** constitutional.

● Core — memory & cache

Source: *_Shared/CORE_02_ComputerOrganization.md*

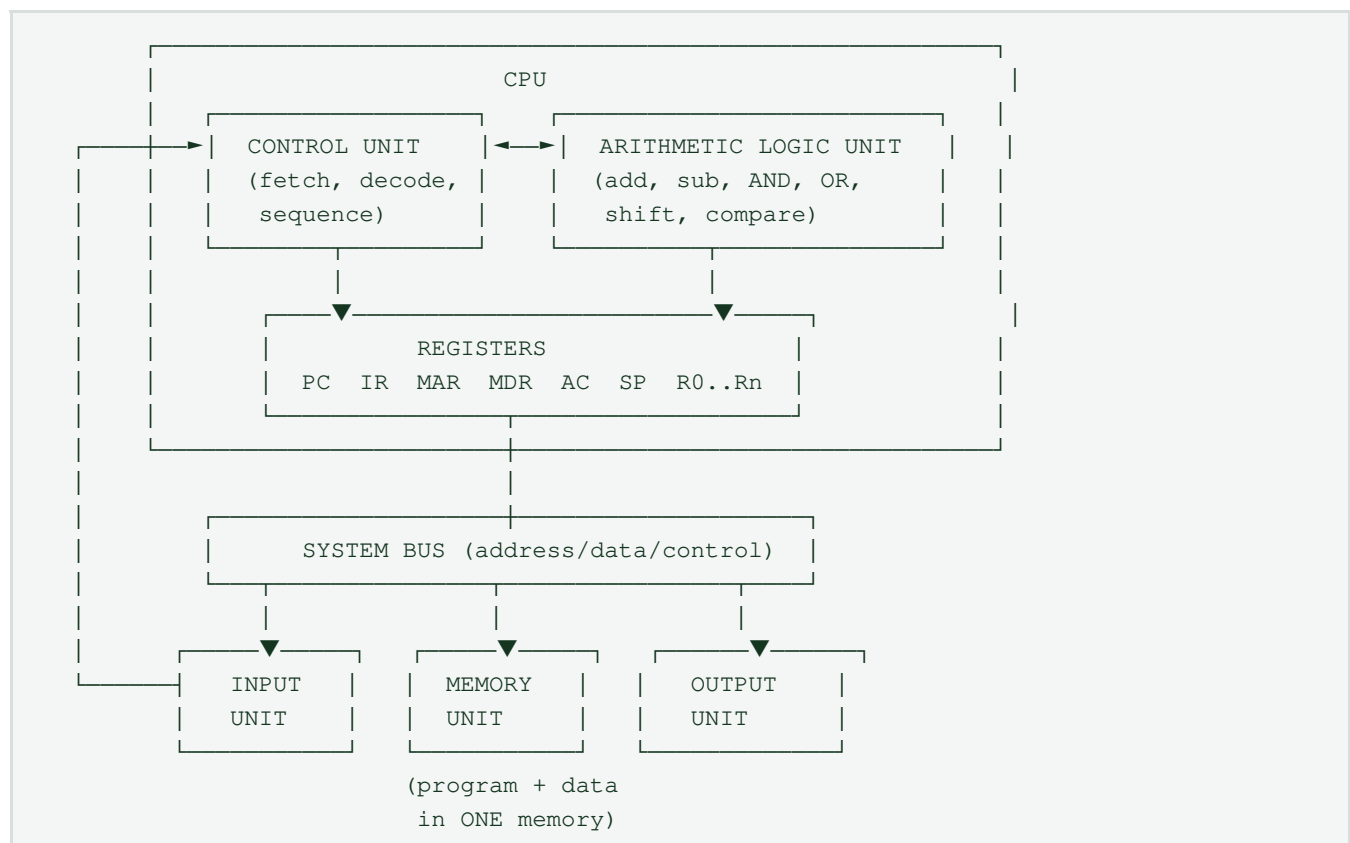
A1. The stored-program concept

Concept — plain English first

Before 1945, "programming" a computer like the ENIAC meant physically rewiring it — days of plugging cables to change what it computed. **John von Neumann's insight (with Eckert and Mauchly) was startlingly simple: instructions are just numbers, so store them in the SAME memory as the data.** Then changing the program is only a matter of loading different numbers into memory. That single idea created the software industry.

The consequence is the structure every general-purpose computer still has: a memory holding both program and data, a processing unit that reads instructions from it one at a time, and a control unit that sequences the whole thing.

The five functional units



Unit	Function
Input unit	Accepts data and programs from the outside world (keyboard, mouse, scanner, disk) and converts them to machine-readable form
Memory unit	Stores the program and data. Primary (RAM/ROM/cache) and secondary
ALU	Performs all arithmetic (+, −, ×, ÷) and logic (AND, OR, NOT, XOR, shift, compare) operations
Control unit	Fetches, decodes and sequences instructions; generates timing and control signals for every other unit. The "nerve centre" — it does no processing itself.
Output unit	Presents results (monitor, printer, plotter, speaker)

CPU = ALU + Control Unit + Registers.

"Central Processing Unit" is CU + ALU + register set; the memory and I/O are outside it.

Key points

1. **Stored-program concept:** instructions and data share one memory, in the same binary

format, and are indistinguishable except by how they are used.

2. **Sequential execution:** instructions execute one after another unless a branch changes the flow. The **Program Counter** holds the address of the next instruction.
3. **The Von Neumann bottleneck:** because there is a *single* bus between CPU and memory, instruction fetches and data accesses must take turns. The CPU can process faster than the bus can supply, so the CPU stalls. **This is the single most important limitation of the model** and the reason caches, pipelining, and Harvard-style split L1 caches exist.

Von Neumann vs Harvard architecture

	Von Neumann	Harvard
Memories	One memory for instructions and data	Separate instruction and data memories
Buses	One shared bus	Two independent buses
Simultaneous fetch of instruction + data	No	Yes
Bottleneck	Yes (the classic bottleneck)	Largely avoided
Cost / complexity	Lower	Higher
Flexibility	Memory freely allocated between code and data	Fixed split
Typical use	General-purpose computers, PCs	DSPs, microcontrollers (PIC, AVR, ARM Cortex-M)
Modified Harvard	Modern CPUs: unified main memory but split L1 instruction and data caches — Von Neumann outside, Harvard inside	

Bus structures

Bus	Carries	Direction	Width determines
Address bus	The memory or I/O address	Unidirectional (CPU → memory)	Addressable memory size = 2^{width} . 16-bit → 64 KB; 20-bit → 1 MB; 32-bit → 4 GB
Data bus	The actual data word	Bidirectional	Word size / transfer size per cycle
Control bus	READ, WRITE, MEMR, MEMW, IOR, IOW, clock, reset, interrupt, bus request/grant	Mixed	—

Single-bus organisation: everything on one bus. Cheap, but only one transfer at a time — slow. **Two-bus / three-bus organisation:** more parallel paths inside the CPU, so a micro-operation like $R3 \leftarrow R1 + R2$ can complete in one clock instead of three.

CPU registers — know every one

Register	Full name	Function
PC	Program Counter (a.k.a. Instruction Pointer)	Holds the address of the NEXT instruction . Incremented during fetch.
IR	Instruction Register	Holds the instruction currently being executed/decoded

MAR	Memory Address Register	Holds the address of the memory location to be accessed. Connects to the address bus. Its size = address bus width.
MDR / MBR	Memory Data (Buffer) Register	Holds the data being read from or written to memory. Connects to the data bus. Its size = data bus width.
AC	Accumulator	Holds one ALU operand and the result (in accumulator-based machines)
SP	Stack Pointer	Holds the address of the top of the stack
Flags / PSW	Status Register / Program Status Word	Condition codes: Z (zero), C (carry), S/N (sign/negative), V/O (overflow), P (parity), plus interrupt-enable and mode bits
General-purpose	R0...Rn	Programmer-visible working registers
TR	Temporary Register	Internal scratch, invisible to the programmer
Index / Base registers	—	Used in indexed and base-relative addressing

A very common MCQ: "Which register is not accessible to the programmer?" — **MAR and MDR** (and TR). PC and SP are visible on most architectures.

Likely exam questions — Von Neumann model

Q	Marks
Explain the stored-program concept.	5
Draw the functional block diagram of a digital computer and describe each unit.	5
List the CPU registers and state the function of each.	5
Compare Von Neumann and Harvard architecture. What is the Von Neumann bottleneck and how do modern processors mitigate it?	10
Explain the basic structural and operational concepts of a computer. Describe the five functional units, the bus organisation (address, data, control), the register set, and explain how the width of the address and data buses limits system capability.	15

MCQ traps — Von Neumann model

Trap	Truth
CPU = ALU + memory	CPU = ALU + CU + registers. Memory is outside.
The control unit performs arithmetic	It only controls ; the ALU computes.
PC holds the current instruction	PC holds the address of the next instruction; IR holds the current instruction.
Address bus is bidirectional	Unidirectional. Only the data bus is bidirectional.
A 32-bit address bus addresses 32 GB	$2^{32} = 4$ GB.
Harvard architecture is obsolete	Universally used in DSPs/microcontrollers and inside every modern CPU's L1 cache.

PART B — INSTRUCTIONS, ADDRESSING AND THE INSTRUCTION CYCLE

C2. Control unit: hardwired vs microprogrammed

Concept

The control unit's job is to emit, at every clock tick, the right set of control signals (`PC_out`, `MAR_in`, `Read`, `ALU_add`, ...) for whatever instruction is in the IR. There are two ways to build the thing that does this:

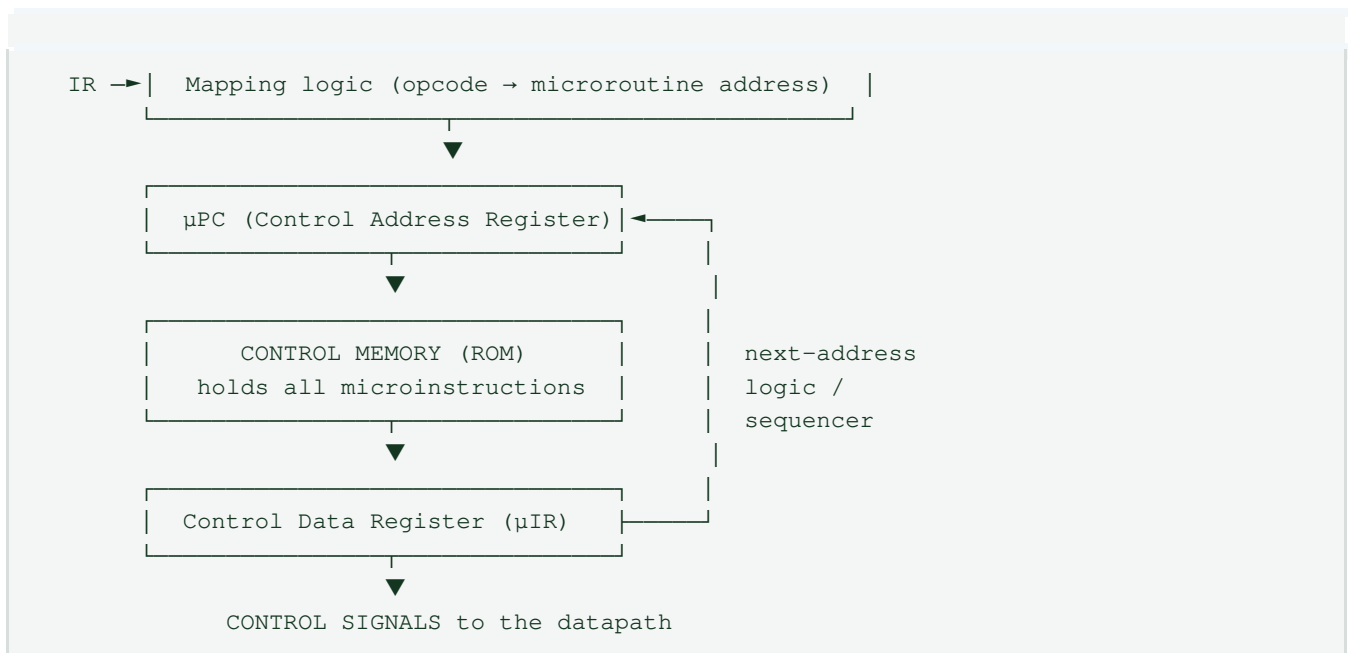
- **Hardwired:** build a fixed combinational/sequential circuit — a state machine of gates and flip-flops — whose outputs *are* the control signals. Fast (it's just logic), but if you want to add an instruction you have to **redesign the hardware**.
- **Microprogrammed (Wilkes, 1951):** store the control signals as **words in a small fast ROM** (the *control memory*). Each word is a **microinstruction** whose bits directly drive the control lines. Executing a machine instruction means running a little program — a **microprogram** — out of that ROM. Slower (an extra memory read per step), but changing the instruction set is now just **rewriting the ROM contents**.

CISC machines (with hundreds of complex instructions) are almost always microprogrammed; RISC machines (few, simple, regular instructions) are hardwired.

Comparison — a guaranteed exam table

Aspect	Hardwired control	Microprogrammed control
Implementation	Fixed logic: gates, decoders, counters, flip-flops (a sequential circuit)	Control memory (ROM) holding microinstructions
Speed	Fast (pure logic delay)	Slower (control-memory read per micro-step)
Flexibility / modification	Very difficult — requires redesigning and refabricating the circuit	Easy — rewrite the microprogram
Design cost / time	High for complex instruction sets	Lower and more systematic
Ability to handle complex instructions	Poor — logic explodes	Good
Chip area	Less (for simple sets)	More (control memory)
Error correction / adding instructions	Hard	Easy
Occurrence	RISC processors	CISC processors
Also called	Random logic control	Firmware control / stored-logic control

Microprogrammed control unit structure



Horizontal vs vertical microprogramming

	Horizontal	Vertical
Encoding	Unencoded — one bit per control signal	Encoded — signals grouped into fields that must be decoded
Microinstruction width	Very wide (hundreds of bits)	Narrow
Parallelism	High — many operations per microinstruction	Low — usually one operation per microinstruction
Control memory size	Large (wide words)	Small (narrow words) but more words needed
Speed	Faster (no decoding step)	Slower (decoder delay)
Microprogram length	Short	Long

Nanoprogramming is a two-level scheme: a vertical microprogram indexes into a horizontal nanoprogram, getting the size advantage of vertical with the parallelism of horizontal.

RISC vs CISC

	CISC	RISC
Instruction count	Large (hundreds)	Small (typically <100)
Instruction length	Variable	Fixed (e.g. 32 bits) — simplifies pipelining
Addressing modes	Many (12–24)	Few (3–5)
Memory access	Almost any instruction can access memory	Only LOAD and STORE ("load-store architecture")
Registers	Few	Many (32+)
Control unit	Microprogrammed	Hardwired

Cycles per instruction	Variable, multi-cycle	Mostly 1 cycle (pipelined)
Pipelining	Difficult	Easy and deep
Code size	Smaller	Larger
Compiler complexity	Simpler compiler, complex hardware	Complex compiler , simple hardware
Examples	x86, VAX, Motorola 68000, IBM 360	ARM, MIPS, SPARC, PowerPC, RISC-V

Modern x86 chips are CISC on the outside but **translate instructions into RISC-like micro-ops internally** — a good closing sentence for a 10-mark answer.

Likely exam questions — CPU and control unit

Q	Marks
What is a micro-operation? Show the micro-operations for $R3 \leftarrow R1 + R2$ on a single-bus CPU.	5
Differentiate between horizontal and vertical microprogramming.	5
Compare hardwired and microprogrammed control units. Draw the block diagram of a microprogrammed control unit.	10
Compare RISC and CISC architectures under at least eight headings.	10
Describe the design of a CPU datapath. Explain single-bus versus three-bus organisation with the micro-operation sequences for an ALU operation in each, and discuss how bus organisation affects the number of clock cycles per instruction.	15

MCQ traps — CPU and control unit

Trap	Truth
Microprogrammed control is faster	Hardwired is faster. Microprogrammed is more flexible .
Horizontal microinstructions are narrow	Horizontal = wide and unencoded ; vertical = narrow and encoded.
RISC has more addressing modes	Fewer.
RISC programs are smaller	Larger code size (more instructions), but each is fast.
CISC is hardwired	CISC is microprogrammed ; RISC is hardwired.
Control memory holds the user's program	It holds microinstructions (firmware), not user code.

PART D — MEMORY ORGANISATION

D1. The memory hierarchy

Concept

You want memory that is **fast**, **big**, and **cheap**. Physics and economics let you pick

two. So instead of one memory, computers use a **pyramid**: a tiny amount of extremely fast memory next to the CPU, backed by progressively larger, slower, cheaper levels.

Why this works at all is one single fact: the principle of locality of reference.

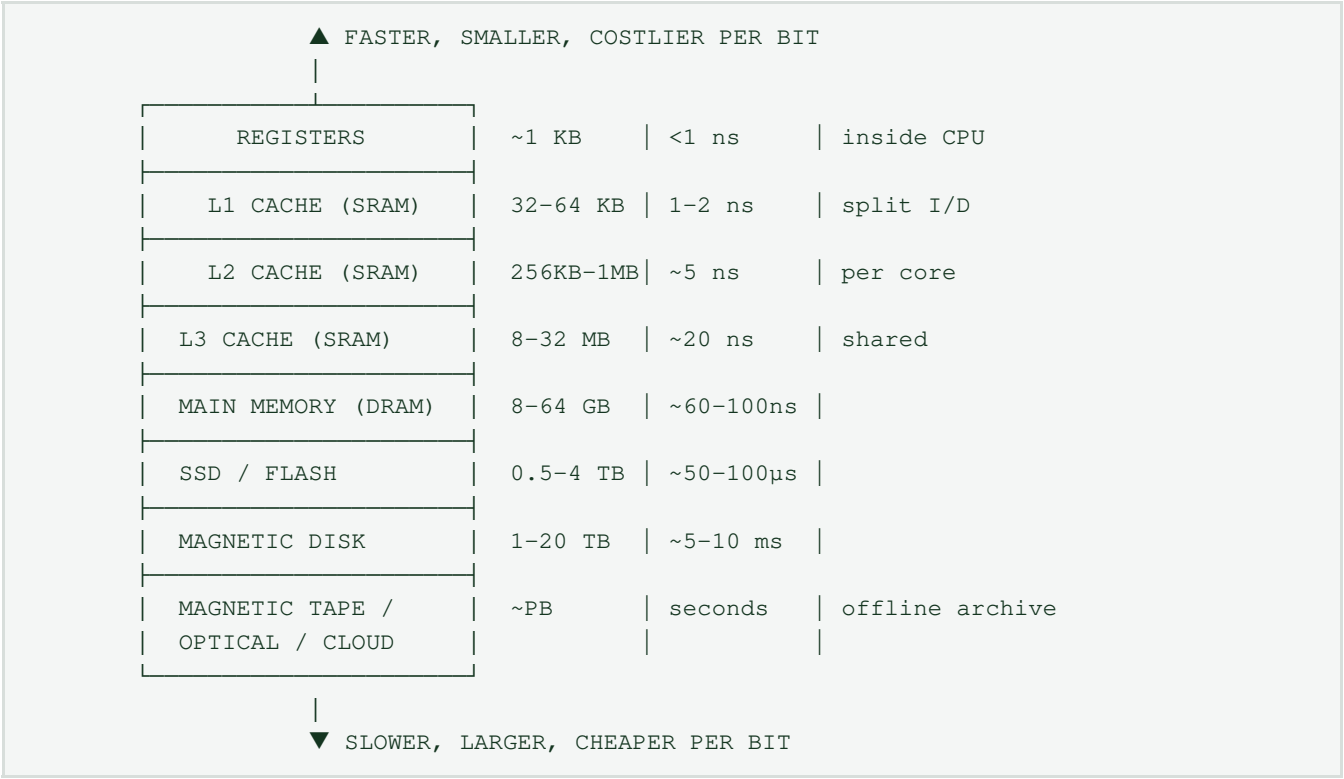
Programs don't touch memory randomly. They touch a small region intensively for a while, then move on. So if you keep the currently-hot region in the fast level, the *overwhelming majority* of accesses are served at the fast level's speed while the *capacity* you appear to have is that of the slow level. That's the whole idea, and it is worth stating in exactly those words in a 15-mark answer.

Locality of reference — the three kinds

Type	Meaning	Everyday cause
Temporal locality	A location referenced now is likely to be referenced again soon	Loop counters, frequently-called functions, stack variables
Spatial locality	Locations near a referenced location are likely to be referenced soon	Sequential instruction fetch, array traversal, struct field access
Sequential locality	A special case of spatial: addresses accessed in increasing order	Instruction streams, file reads

Spatial locality is why caches fetch a whole **block/line** rather than a single word.
Temporal locality is why we keep blocks around instead of discarding them immediately.

The hierarchy



The ratios to quote: registers are roughly **100×** faster than DRAM; DRAM is roughly **100,000×** faster than disk. That million-fold gap between RAM and disk is why a page fault is catastrophic and a cache miss is merely annoying.

Level	Managed by	Transfer unit

Registers ↔ Cache	Compiler / hardware	Word
Cache ↔ Main memory	Hardware (cache controller)	Block / line (typically 32–64 bytes)
Main memory ↔ Disk	Operating system (virtual memory)	Page (typically 4 KB)
Disk ↔ Tape/archive	User / OS	File

D2. RAM and ROM types

RAM: SRAM vs DRAM

	SRAM (Static RAM)	DRAM (Dynamic RAM)
Storage element	Flip-flop (6 transistors)	Capacitor + 1 transistor
Refresh needed	No	Yes — the capacitor leaks; must be refreshed every few milliseconds
Speed	Fast (~1–10 ns)	Slower (~50–70 ns)
Density	Low (big cell)	High (tiny cell)
Cost per bit	High	Low
Power	Higher static, lower during access	Lower static, but refresh consumes power
Volatile?	Yes (both are volatile)	Yes
Used for	Cache memory , registers	Main memory

The single most-asked RAM MCQ: which needs refreshing? → DRAM, because it stores charge on a capacitor that leaks.

DRAM generations (Diploma syllabus names SDRAM vs DDR explicitly)

Type	Full name	Key property
FPM / EDO DRAM	Fast Page Mode / Extended Data Out	Legacy asynchronous DRAM
SDRAM	Synchronous DRAM	Synchronised to the system clock; transfers once per clock cycle (on the rising edge only)
DDR SDRAM	Double Data Rate	Transfers on BOTH rising and falling clock edges → 2× the bandwidth at the same clock
DDR2		4 transfers per clock at the I/O bus (2× DDR), lower voltage (1.8 V)
DDR3		8 transfers per clock, 1.5 V
DDR4		16 transfers per clock, 1.2 V, higher density
DDR5		Higher speed again, on-die ECC, 1.1 V

RDRAM	Rambus DRAM	Proprietary high-speed, largely obsolete

SDRAM vs DDR — the exam answer in one line: *SDRAM transfers data once per clock cycle (rising edge only); DDR transfers on both edges, doubling the data rate without increasing the clock frequency.*

Also know: **DIMM** = Dual In-line Memory Module (independent contacts on both sides, 64-bit data path — the modern standard). **SIMM** = Single In-line (contacts on both sides are electrically the same, 32-bit). **SO-DIMM** = the small laptop version.

ROM family — memorise this table, it is pure Diploma MCQ material

Type	Full name	How written	How erased	Reusable?
ROM (Mask ROM)	Read Only Memory	At the factory , during fabrication (masked in)	Cannot be erased	No
PROM	Programmable ROM	Once by the user , using a PROM programmer that blows fusible links	Cannot be erased	No — one-time programmable (OTP)
EPROM	Erasable PROM	Electrically, by a programmer	Ultraviolet light through a quartz window (~20 min), erases the whole chip	Yes, but must be removed from the circuit
EEPROM	Electrically Erasable PROM	Electrically	Electrically, byte by byte , in-circuit	Yes
Flash	Flash EEPROM	Electrically	Electrically, in BLOCKS (faster than EEPROM's byte-wise erase)	Yes — the basis of SSDs, USB drives, BIOS

- All ROM types are **non-volatile** (retain data without power) and **random access**.
- Uses: **BIOS/firmware**, bootstrap loader, lookup tables, embedded programs.
- **NOR flash** allows random read (used for code/BIOS); **NAND flash** is denser and read in pages (used for SSDs and memory cards).

Memory chip organisation and address decoding

A memory chip described as "**1K × 8**" means **1024 locations of 8 bits each**:

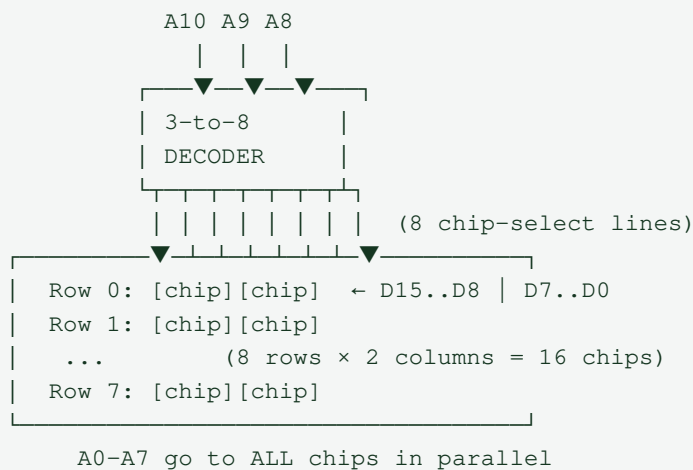
- Address lines needed: $2^{10} = 1024 \rightarrow$ **10 address lines**
- Data lines: **8**
- Total capacity: 1 KB

Worked example 3 — chip-count problem (a very common numerical).

How many 256×8 RAM chips are needed to build a $2K \times 16$ memory system? How many address lines does the system need and how many are decoded?

```
Chips needed vertically (capacity):  2048 / 256 = 8 rows
Chips needed horizontally (width):    16 bits / 8 bits = 2 columns
TOTAL CHIPS = 8 × 2 = 16
```

```
Each chip has 256 locations → 28 → 8 address lines go to every chip (A0-A7)
The system has 2048 locations → 211 → 11 address lines total (A0-A10)
The remaining 3 lines (A8, A9, A10) select WHICH ROW of chips →
    decoded by a 3-to-8 decoder driving the 8 chip-select (CS) lines
```



Linear selection (linear decoding): instead of a decoder, use one high-order address line directly as each chip's CS. Cheap (no decoder) but **wasteful of address space** and you must never assert two CS lines at once. **Full decoding** uses a decoder and utilises the entire address space — this is the "linear selection technique of I/O addressing" mentioned in the CF syllabus.

D3. Cache memory — THE key topic

Concept

The CPU is far faster than DRAM. Every time it waits for main memory it wastes hundreds of cycles doing nothing. **Cache is a small, very fast SRAM buffer that sits between the CPU and main memory and holds copies of the memory blocks currently in use.**

The mechanism: when the CPU asks for an address, the cache is checked first. If the data is there — a **hit** — it is delivered at cache speed. If not — a **miss** — the whole **block** containing that address is copied from main memory into the cache (bringing its neighbours along, exploiting spatial locality) and then delivered. Because of locality, hit rates of 95–99% are routine, so the *average* access time is close to the cache's speed even though the *capacity* is main memory's.

The entire design problem is: **where in the cache may a given main-memory block be placed, and how do we know quickly whether it is there?** The three answers are the three mapping techniques.

Terminology

Term	Meaning
Block / Line	The unit of transfer between main memory and cache (e.g. 16 bytes)
Line/Slot	A place in the cache that can hold one block
Tag	Extra bits stored with each cache line identifying which main-memory block it holds
Valid bit	Says whether the line currently holds real data
Dirty bit	Says the line has been modified and must be written back
Hit ratio (h)	hits / total accesses

Miss ratio	$1 - h$
Miss penalty	The extra time to service a miss

The three mapping techniques

1. Direct mapping

Concept: each main-memory block has **exactly one** cache line it is allowed to occupy, determined by `block number MOD number of cache lines`. Dead simple and dead fast to check — you go straight to the one possible line and compare one tag. The cost: if a program alternates between two blocks that map to the same line, they evict each other on every access — **thrashing / conflict misses** — even if the rest of the cache is empty.

Address split:

TAG	LINE (index)	WORD/OFFSET
-----	--------------	-------------

2. Fully associative mapping

Concept: a block may go in **any** cache line. Maximum flexibility, so no conflict misses at all — a block is only evicted when the whole cache is full. The cost: to find out whether a block is present you must compare its tag against **every** line's tag. That needs expensive parallel comparison hardware (**content-addressable memory**), which is why fully associative caches are only used when they are small (e.g. TLBs).

Address split:

TAG	WORD/OFFSET
-----	-------------

3. Set-associative mapping — the practical compromise

Concept: divide the cache into **sets** of k lines each ("k-way set associative"). A block maps to exactly one **set** (like direct mapping) but may sit in **any line within that set** (like associative). You only compare k tags, which is cheap, and you get most of the conflict-miss benefit. **Every real CPU cache is set-associative**, typically 4-way, 8-way or 16-way.

Address split:

TAG	SET	WORD/OFFSET
-----	-----	-------------

Note the special cases: **1-way set associative = direct mapped**; **n-way where n = number of lines = fully associative**.

Comparison table

	Direct	Fully associative	k-way set associative
A block can go in	1 specific line	Any line	Any of k lines in one set
Tag comparators needed	1	All lines (parallel)	k

Hardware cost	Lowest	Highest	Moderate
Search speed	Fastest	Slowest / most expensive	Fast
Conflict misses	Many (thrashing)	None	Few
Replacement algorithm needed?	No (only one choice)	Yes	Yes (within the set)
Tag field size	Smallest	Largest	Middle
Used in practice	Simple/embedded	TLBs, small buffers	All modern CPU caches

Worked example 4 — address partitioning for all three schemes

Given: Main memory = 64 KB (byte addressable). Cache = 2 KB. Block size = 16 bytes.

Find the address format for direct, associative and 4-way set-associative mapping.

STEP 1 – Basic quantities

Physical address size = $\log_2(64 \text{ K}) = \log_2(2^{16}) = 16 \text{ bits}$

Number of blocks in main memory = $64 \text{ KB} / 16 \text{ B} = 4096 = 2^{12}$

Number of lines in cache = $2 \text{ KB} / 16 \text{ B} = 128 = 2^7$

Word/offset bits = $\log_2(16) = 4 \text{ bits}$ ← same for all three schemes

(a) DIRECT MAPPING

Line (index) bits = $\log_2(128) = 7$

Tag bits = $16 - 7 - 4 = 5$

TAG 5	LINE 7	WORD 4
-------	--------	--------

= 16 bits ✓

Main memory block j maps to cache line $(j \bmod 128)$.

(b) FULLY ASSOCIATIVE MAPPING

Tag bits = $16 - 4 = 12$ (the tag is the full block number)

TAG 12	WORD 4
--------	--------

= 16 bits ✓

Needs 128 parallel 12-bit comparators.

(c) 4-WAY SET ASSOCIATIVE MAPPING

Number of sets = $128 \text{ lines} / 4 \text{ lines per set} = 32 = 2^5$

Set bits = 5

Tag bits = $16 - 5 - 4 = 7$

TAG 7	SET 5	WORD 4
-------	-------	--------

= 16 bits ✓

Main memory block j maps to set $(j \bmod 32)$, any of its 4 lines.

Summary table to reproduce:

Scheme	Tag	Line/Set	Word	Comparators
Direct	5	7 (line)	4	1

3	1 2 3	Miss
4	1 2 3 4	Miss
1	1 2 3 4	Hit
2	1 2 3 4	Hit
5	2 3 4 5 (evict 1)	Miss
1	3 4 5 1 (evict 2)	Miss
2	4 5 1 2 (evict 3)	Miss
3	5 1 2 3 (evict 4)	Miss
4	1 2 3 4 (evict 5)	Miss
5	2 3 4 5 (evict 1)	Miss

FIFO: 10 misses, 2 hits.

LRU:

Ref	Cache (LRU → MRU)	Hit/Miss
1	1	Miss
2	1 2	Miss
3	1 2 3	Miss
4	1 2 3 4	Miss
1	2 3 4 1	Hit
2	3 4 1 2	Hit
5	4 1 2 5 (evict 3, the LRU)	Miss
1	4 2 5 1	Hit
2	4 5 1 2	Hit
3	5 1 2 3 (evict 4)	Miss
4	1 2 3 4 (evict 5)	Miss
5	2 3 4 5 (evict 1)	Miss

LRU: 8 misses, 4 hits. LRU wins because it recognises that 1 and 2 are being reused.

Write policies

Policy	Mechanism	Advantage	Disadvantage
Write-through	Every write goes to both cache and main memory immediately	Main memory is always consistent ; simple; good for multiprocessors and DMA	High memory traffic — every write costs a slow memory access

Write-back (copy-back)	Write only to the cache; set a dirty bit ; write to memory only when the block is evicted	Much less memory traffic : multiple writes to the same block cost one memory write	Memory is temporarily inconsistent (a <i>cache coherence</i> problem for DMA and other CPUs); more complex
-------------------------------	---	---	---

On a write miss:

- **Write-allocate (fetch-on-write)**: load the block into cache first, then write. Usually paired with write-back.
- **No-write-allocate (write-around)**: write straight to memory, don't load the block. Usually paired with write-through.

Cache performance numericals

The two formulas — know which one the question wants:

```
SIMULTANEOUS / PARALLEL access (cache and memory searched at the same time):
    T_avg = h · Tc + (1 - h) · Tm

HIERARCHICAL / SEQUENTIAL access (memory searched only AFTER a cache miss):
    T_avg = h · Tc + (1 - h) · (Tc + Tm)
           = Tc + (1 - h) · Tm

where h = hit ratio, Tc = cache access time, Tm = main memory access time
```

If a question doesn't specify, **state your assumption** and use the hierarchical form (it is the more realistic and more commonly intended one) — or give both.

Worked example 7 — basic average access time.

A system has cache access time 10 ns, main memory access time 100 ns and a hit ratio of 0.90. Find the average access time under both models. What is the speedup over a cache-less system?

```
SIMULTANEOUS:
T_avg = 0.90 × 10 + 0.10 × 100 = 9 + 10 = 19 ns

HIERARCHICAL:
T_avg = 0.90 × 10 + 0.10 × (10 + 100) = 9 + 11 = 20 ns
(equivalently 10 + 0.10 × 100 = 20 ns)

Without any cache, every access costs 100 ns.
Speedup (hierarchical) = 100 / 20 = 5×
```

Worked example 8 — find the required hit ratio.

Cache 10 ns, memory 100 ns. What hit ratio is needed for an average access time of 15 ns (hierarchical model)?

```
15 = 10 + (1 - h) (100)
5 = 100 - 100h
100h = 95
h = 0.95 → a 95% hit ratio is required
```

Worked example 9 — hit ratio from a trace.

A program makes 1000 memory references; 60 of them miss in the cache. Cache = 20 ns, memory = 150 ns (hierarchical). Find hit ratio, miss ratio and average access time.

```
Hits    = 1000 - 60 = 940
h       = 940 / 1000 = 0.94   (94%)
Miss    = 0.06
T_avg   = 0.94 × 20 + 0.06 × (20 + 150)
         = 18.8 + 10.2 = 29 ns
```

Worked example 10 — MULTI-LEVEL cache (the harder version, often 15 marks).

A system has L1 cache (access 1 ns, hit ratio 0.80), L2 cache (access 10 ns, hit ratio 0.95 for references that reach it) and main memory (access 100 ns). Accesses are hierarchical. Find the effective access time.

Work OUTWARD from the CPU:

```
Every access pays the L1 access time:           1 ns
20% of accesses (miss L1) go on to L2:         0.20 × [L2 cost]
    L2 cost = L2 access + (L2 miss) × memory
              = 10 + 0.05 × 100
              = 10 + 5 = 15 ns
T_eff = 1 + 0.20 × 15 = 1 + 3 = 4 ns
```

Global miss rate = fraction of ALL references that reach main memory
 $= 0.20 \times 0.05 = 0.01 = 1\%$.

Local miss rate of L2 = 0.05 = 5%. (Distinguishing local from global miss rate is a favourite follow-up.)

Worked example 11 — the "how much faster" question.

A processor takes 1 cycle per instruction when all accesses hit the cache. The miss penalty is 50 cycles, the miss rate is 4%, and 30% of instructions also make a data reference. Find the effective CPI.

```
Memory accesses per instruction = 1 (instruction fetch) + 0.30 (data) = 1.30
Stall cycles per instruction = 1.30 × 0.04 × 50 = 2.6
Effective CPI = 1 + 2.6 = 3.6
```

The machine runs 3.6× slower than the ideal — a good illustration of why cache performance dominates real system performance.

Types of cache miss — the "three Cs" (a neat 5-mark answer)

Miss type	Cause	Cure
Compulsory (cold-start)	The very first reference to a block — it has never been in the cache	Larger blocks; prefetching
Capacity	The working set is larger than the whole cache	Larger cache
Conflict (collision)	Too many active blocks map to the same set, even though the cache isn't full	Higher associativity
(Coherence)	In multiprocessors, a block invalidated by another CPU's write	Coherence protocol tuning

Likely exam questions — Cache